



ADR-002: Node.js 22, minimal dependencies (`pg` only at runtime), JavaScript with JSDoc

TASCO Insurance × VETC Growth Platform — Architecture Decision Record

iorta TechNXT

Version 1.0 · 07 October 2026

Contents

Context and problem statement

Decision drivers

Considered options

Decision outcome

Pros and cons of the options

Document	Version	Date	Owner	Status
ADR-002	1.0	07 October 2026	iorta TechNXT Architecture	Accepted

- **Related:** [ADR-001](#), [ADR-010](#), [ADR-011](#)

Context and problem statement

The platform processes personal data for about 6 M vehicle owners and moves money through the VETC wallet. Software supply-chain compromise is a top risk: typosquatting, maintainer takeover, transitive vulnerabilities. Insurers and their regulators expect a defensible SBOM and patch cadence. The team also needs fast delivery and easy onboarding.

Decision drivers

- Minimise attack surface and the transitive dependency count.
- Use an LTS runtime with a long support window.
- Fast startup and low memory for containers and CronJobs.
- Keep a single language across server, jobs and SPA.

Considered options

1. Node.js 22 LTS, standard library first, `pg` as the only runtime dependency, JavaScript with JSDoc types.
2. Node.js + TypeScript + framework stack (Express/Nest, Prisma/TypeORM, Joi/Zod, jsonwebtoken, prom-client).
3. Java / Spring Boot.

Decision outcome

Chosen option: **1.** `package.json` declares `"engines": {"node": "≥ 22.12"}` and `"dependencies": {"pg": "^8.23.1"}`. Dev dependencies are lint-only (`eslint`, `@eslint/js`, `globals`).

Capabilities are built on `node:*` built-ins:

Capability	Implementation
HTTP server, routing	<code>node:http</code> , <code>src/adapters/http/router.js</code> (about 40 lines)
Crypto (AES-GCM, HMAC, scrypt, JWT HS256, TOTP)	<code>node:crypto</code> , <code>src/shared/crypto.js</code>
Validation (allow-list)	<code>src/shared/validation.js</code>
Metrics (Prometheus text format)	<code>src/shared/metrics.js</code>
Tests and coverage	<code>node --test</code> , <code>--experimental-test-coverage</code> with gates (lines 80 %, branches 70 %, functions 80 %)
<code>pg</code>	Loaded lazily in <code>postgresStore.js</code> , so in-memory mode has zero runtime dependencies

Consequences

- Good: a very small SBOM, `npm audit --omit=dev` stays meaningful, and Dependabot/Trivy noise stays low.
- Good: hand-written primitives are short enough to review line by line. Each one is pinned to a standard: RFC 7519 (JWT), RFC 6238 (TOTP) and OWASP ASVS. JWT verification pins `alg` to HS256 and uses a constant-time comparison.
- Bad: home-grown security primitives carry implementation risk. **Mitigations:**
 - unit tests with RFC test vectors (TOTP);
 - external pen-test scope includes the auth module;
 - [ADR-007](#) moves token issuance to the IdP (OIDC), which retires the local JWT signer for staff.
- Bad: there is no compile-time type safety. **Mitigations:** JSDoc on public functions, ESLint, high test coverage, and an option to enable `// @ts-check + tsc --noEmit` in CI without changing the runtime.
- Bad: features that frameworks provide for free are missing: streaming body parsing, content negotiation, HTTP/2. These are acceptable for a JSON API behind an ingress.

Pros and cons of the options

TypeScript and frameworks. Better ergonomics and typing. However, they add hundreds of transitive packages, increasing supply-chain and patching load.

Java / Spring. Mature in insurance IT. However, it brings a heavier runtime, slower iteration for the pilot, and a different skill set from the SPA.